FIAP

CP05 - Pipeline DevSecOps

DevSecOps and Cloud Computing

Nome: Joao Lucas Yudi

RM: [RM]

Repositorio: [link publico apos o push]

Os paragrafos abaixo estao marcados como rascunho. Reescrever com as proprias palavras antes de entregar.

1. Visao geral

O repositorio publica um site estatico (nginx) e um workflow chamado Security Pipeline. Ele dispara em todo push e em todo pull request, com tres jobs de nomes fixos: gitleaks, semgrep e trivy. A ideia de Shift Left e colocar essas barreiras no momento do commit e do pull request, antes do codigo chegar na branch principal e muito antes de ir para producao.

Cada sabotagem ficou numa branch propria (sabotage/gitleaks, sabotage/semgrep, sabotage/trivy) e nao entra na main. A main fica com a imagem remediada.

2. Barreira 1: Gitleaks

Print do job gitleaks falhando na branch sabotage/gitleaks

COLAR O PRINT AQUI
Aba Actions, job gitleaks, status failed.
O print precisa mostrar o username do GitHub, o nome do repositorio e o horario.
De preferencia o trecho do log com RuleID aws-access-token, o arquivo config.js e o commit.

RASCUNHO, reescrever

Eu coloquei uma chave falsa de acesso da AWS num arquivo config.js, num commit da branch sabotage/gitleaks, e o job gitleaks falhou. A regra que disparou foi aws-access-token, no arquivo config.js. Isso importa porque bots varrem repositorios publicos em minutos: uma chave real seria usada antes de eu perceber. Apagar o arquivo num commit seguinte nao resolve, porque o Git guarda o historico e o scan com fetch-depth 0 le todos os commits, nao so o ultimo. A solucao real, se a chave fosse verdadeira, seria revoga-la no provedor. O fetch-depth 0 e obrigatorio: sem o historico completo o scanner so olha o snapshot atual e a chave antiga passa.

3. Barreira 2: Semgrep

Print da finding de eval no job semgrep

COLAR O PRINT AQUI
Aba Actions, job semgrep, finding em vulnerable.js.
Mostrar username, nome do repositorio e horario.
No log: regra javascript.browser.security.eval-detected.eval-detected, arquivo e linha 3.

RASCUNHO, reescrever

No arquivo vulnerable.js a funcao calcular devolve eval da entrada do usuario. O Semgrep apontou a regra javascript.browser.security.eval-detected.eval-detected, na linha 3, como finding bloqueante. eval com dado que eu nao controlo e injecao de codigo: quem chama a funcao escolhe o que o processo executa. Isso cai na classe Injection, OWASP A03. O principio e nunca confiar na entrada do usuario. O Semgrep e analise estatica: ele le o codigo sem executar o programa e ainda assim ve o caminho perigoso.

Nota de configuracao, para eu lembrar na reescrita: o slide usa semgrep ci dentro da imagem semgrep/semgrep. Essa imagem nao tem Node, entao o checkout da Action nao roda la, e o semgrep ci pede o contexto do Semgrep App. O pacote p/javascript nao viu o eval. O p/default viu, mas tambem reclamava das tags do proprio workflow, que o enunciado pede. Por isso o job roda no ubuntu-latest, com semgrep scan --config p/default --error, excluindo a pasta .github.

4. Barreira 3: Trivy

Print do scan da imagem na main e, se couber, do job falhando em sabotage/trivy

COLAR O PRINT AQUI
Job trivy verde na main (0 HIGH/CRITICAL depois da remediacao).
Opcional: job trivy vermelho na branch sabotage/trivy, imagem nginx:1.14.
Username, nome do repositorio e horario visiveis.

RASCUNHO, reescrever

O Trivy olha a imagem que o proprio projeto constroi: o sistema da imagem base e os pacotes que vao junto com o nginx. Uma CVE e uma vulnerabilidade publicada; o CVSS da uma nota de 0 a 10, e acima de 9 o achado e critico. O job so falha em HIGH e CRITICAL, por causa do exit-code 1: sem isso o scan imprimiria o problema e o pipeline seguiria verde.

No baseline local, nginx:1.30.4-alpine tinha 10 achados HIGH e nenhum CRITICAL. Os pacotes eram libexpat (CVE-2026-93990), libuuid do util-linux (CVE-2026-53612, CVE-2026-53613, CVE-2026-53614, CVE-2026-76642, CVE-2026-78408, CVE-2026-78409 e CVE-2026-78410), pcre2 (CVE-2026-103111) e tiff (CVE-2026-4775). Trocar so a tag para nginx:1.31.6-alpine baixou para 3 HIGH, porque o Alpine da imagem nova ainda nao tinha libexpat, pcre2 e tiff corrigidos. O passo que zerou o scan foi o apk upgrade em cima de nginx:1.31.6-alpine. O depois local ficou em 0 HIGH e 0 CRITICAL, e e esse Dockerfile que esta na main.

A branch sabotage/trivy troca a base para nginx:1.14, uma imagem antiga citada na aula, so para mostrar o job barrando CVEs. Ela nao e mergeada. No scan local dessa imagem (Debian 9.8) o Trivy apontou 113 achados na faixa pedida: 82 HIGH e 31 CRITICAL.

5. Conclusao

RASCUNHO, reescrever

As tres ferramentas cobrem pontos cegos diferentes. O Gitleaks ve segredo no historico do Git, coisa que o Semgrep nao foi feito para caçar e que o Trivy nao ve se o segredo nem entrou na imagem. O Semgrep le o codigo da aplicacao e pega um eval perigoso que nao e nem credencial nem pacote velho. O Trivy olha o que a imagem leva para producao, mesmo quando o meu HTML esta limpo. Juntas, elas sao defesa em profundidade: uma barreira que passa nao significa que as outras passam.

6. Dados para consultar na hora do print

Preencher as URLs e os horarios depois que os runs existirem. A lista completa esta em evidencias.md.

- main: commit 0505fb2. Gitleaks e Semgrep devem passar. Trivy deve passar (0 HIGH/CRITICAL).

- sabotage/gitleaks: commit do segredo 0bf877e e commit que apaga o arquivo 7be0924. O job gitleaks deve continuar failed.

- sabotage/semgrep: commit 19ac102, vulnerable.js, regra eval-detected, linha 3.

- sabotage/trivy: commit b367cc6, FROM nginx:1.14. O job trivy deve falhar.

- fix/trivy-base-image aponta para o mesmo commit da main (0505fb2), que e a remediacao.

- RM da capa ainda esta como [RM]. Link do repositorio entra depois do push.